Glossary & Acronyms

Core Technologies

ASGI

Asynchronous Server Gateway Interface

Python standard for async web servers/frameworks Successor to WSGI (Web Server Gateway Interface) Enables async/await in web applications Used by: FastAPI, Starlette, Django Channels, Quart **Example servers:**
- Uvicorn (fastest)
- Hypercorn
- Daphne

vs WSGI:
- WSGI = synchronous (older, Flask/Django)
- ASGI = asynchronous (modern, supports WebSocket)

WSGI

Web Server Gateway Interface

Python standard for synchronous web servers Older specification (PEP 3333) Used by: Flask, Django (traditional), Pyramid **Example servers:**
- Gunicorn
- uWSGI
- mod_wsgi

Limitation: No WebSocket support (synchronous only)

Goroutine

Go's lightweight thread

Managed by Go runtime (not OS) ~2KB memory per goroutine Scheduled cooperatively (M:N threading) Can have millions in single process **vs OS Thread:**
- OS Thread: ~2MB, heavy context switch
- Goroutine: ~2KB, cheap context switch

Event Loop

Single-threaded async execution model

Core of Python's asyncio Handles multiple I/O operations concurrently Tasks yield control with `await` One blocking call = entire loop blocked **Languages using event loop:**
- Python (asyncio)
- JavaScript (Node.js)
- Ruby (EventMachine)

GIL

Global Interpreter Lock

Python's memory safety mechanism Only one thread executes Python code at a time Limits CPU-bound parallelism Doesn't affect I/O-bound async code **Impact:**
- Multi-threading doesn't help CPU-bound tasks
- Use multiprocessing for CPU parallelism
- Not present in Go, Rust, Java

Web Frameworks

FastAPI

Modern Python web framework

Built on ASGI (Starlette + Pydantic) Automatic API documentation (OpenAPI) Type hints for validation Async/await support **Key features:**
- Dependency injection
- Automatic validation
- OAuth2/JWT support
- WebSocket support

Gin

Go web framework

Fast HTTP router Middleware support JSON validation Low memory footprint **Alternatives:**
- Echo (similar to Gin)
- Fiber (Express-like)
- Chi (lightweight)

Uvicorn

ASGI server for Python

Fast async web server Uses uvloop (libuv wrapper) Production-ready Typically run with 4-8 workers **Usage:**

```
uvicorn app:app --workers 4
```

Monitoring & Observability

APM

Application Performance Monitoring

Tools: New Relic, Datadog, Dynatrace Tracks: Response time, errors, throughput Distributed tracing Real user monitoring (RUM) Prometheus

Open-source monitoring system

Time-series database Pull-based metrics collection PromQL query language Industry standard for Kubernetes **Metrics types:**
- Counter (increments only)
- Gauge (goes up/down)
- Histogram (bucketed observations)
- Summary (quantiles)

OpenTelemetry

Observability framework

Traces, metrics, logs unified Vendor-neutral Successor to OpenTracing + OpenCensus Works with: Jaeger, Zipkin, Prometheus pprof

Go profiling tool

Built into Go runtime CPU profiling Memory profiling Goroutine profiling Block profiling **Access:**

```
import _ "net/http/pprof"
// http://localhost:6060/debug/pprof/
```

Security

JWT

JSON Web Token

Stateless authentication Self-contained (claims embedded) Signed (HMAC or RSA) Used for: API auth, SSO **Structure:** `header.payload.signature`

OAuth2

Authorization framework

Delegated authorization Access tokens (short-lived) Refresh tokens (long-lived) Flows: Authorization code, Client credentials, etc. CSRF

Cross-Site Request Forgery

Attack: Unauthorized actions on behalf of user Protection: CSRF tokens SameSite cookies Required for: Form submissions CORS

Cross-Origin Resource Sharing

Browser security feature Controls which origins can access API Headers: Access-Control-Allow-Origin, etc. Required for: Frontend → Backend on different domains Architecture Patterns

Microservices

Distributed architecture

Small, independent services Each owns its data Communicate via API/events Independently deployable **vs Monolith:**
- Monolith: Single codebase/deployment
- Microservices: Multiple services

API Gateway

Entry point for microservices

Routes requests to services Authentication/authorization Rate limiting Load balancing **Examples:**
- Kong
- Traefik
- AWS API Gateway

Job Queue

Async task processing

Decouple request from processing Background jobs Retry logic Priority queues **Technologies:**
- Redis (simple)
- RabbitMQ (robust)
- AWS SQS (managed)

Concurrency Models

Async/Await

Cooperative multitasking

Single-threaded Explicit yielding (await) Non-blocking I/O Used by: Python, JavaScript, C#
Example:

```
async def fetch():
    result = await http.get(url)  # Yields here
    return result
```

M:N Threading

Many goroutines to N threads

Go runtime scheduler Goroutines multiplexed onto threads Automatic preemption CPU-efficient
Model:
- M goroutines
- N OS threads (usually = CPU cores)
- Go scheduler manages mapping

Green Threads

User-space threads

Managed by language runtime Lighter than OS threads Examples: Go goroutines, Erlang processes **vs OS Threads:**
- Green: 2KB, managed by runtime
- OS: 2MB, managed by kernel

Deployment

Container

Isolated application environment

Docker (most common) Contains: App + dependencies Portable across environments Kubernetes (K8s)

Container orchestration

Automates deployment Scaling Load balancing Self-healing Load Balancer

Distributes traffic

Round-robin, least connections, etc. Health checks SSL termination **Types:**
- Layer 4 (TCP/UDP)
- Layer 7 (HTTP/HTTPS)

Database

PostgreSQL

Relational database

ACID compliant SQL standard JSON support (JSONB) Used in this project Redis

In-memory data store

Key-value store Pub/Sub messaging Job queues Caching **Use cases:**
- Session storage
- Rate limiting
- Job queues (our project)

Performance Metrics

RPS

Requests Per Second

Throughput metric How many requests handled Example: 10,000 RPS Latency

Response time

p50: Median (50th percentile) p95: 95th percentile p99: 99th percentile (tail latency) **Example:** p99 = 100ms means 99% of requests < 100ms

Concurrent Connections

Simultaneous connections

WebSocket: Long-lived connections HTTP: Usually short-lived Go handles 10K+, Python ~1K per worker Common Abbreviations

Term Meaning ASGI Asynchronous Server Gateway Interface WSGI Web Server Gateway Interface JWT JSON Web Token CORS Cross-Origin Resource Sharing CSRF Cross-Site Request Forgery APM Application Performance Monitoring RPS Requests Per Second TLS Transport Layer Security SSL Secure Sockets Layer (deprecated) API Application Programming Interface REST Representational State Transfer CRUD Create, Read, Update, Delete CI/CD Continuous Integration/Deployment DB Database ORM Object-Relational Mapping SQL Structured Query Language NoSQL Not Only SQL JSON JavaScript Object Notation YAML YAML Ain't Markup Language ENV Environment (variables) K8s Kubernetes (8 letters between K & S) VM Virtual Machine CPU Central Processing Unit RAM Random Access Memory I/O Input/Output SDK

Software Development Kit CLI Command Line Interface UI User Interface UX User Experience MVP Minimum Viable Product DX Developer Experience SLA Service Level Agreement SLO Service Level Objective TTL Time To Live DTO Data Transfer Object UUID Universally Unique Identifier EOF End Of File RTFM Read The Fine Manual Project-Specific Terms

Creative Automation Hub

Our project

AI-powered content generation Hybrid Go + Python architecture Real-time WebSocket updates Batch job processing Text Generation

AI text content creation

Uses: Groq (fast) or Anthropic (quality) Types: Blog, social, ad copy Multi-variant: 1-10 versions Tone customization Image Generation

AI image creation

Stable Diffusion API Text-to-image Style presets Batch generation Brand Kit

Visual identity storage

Colors (hex codes) Fonts Logo URL Applied to all generated content Job Queue

Our Redis-based queue

Producer: Go backend Consumer: Python workers Queues: `queue:text`, `queue:image` Status: pending → processing → completed/failed Quick

Reference

When you see:
- **ASGI** → Think: Python async web server standard
- **Goroutine** → Think: Go's lightweight thread
- **Event Loop** → Think: Single-threaded async (Python/JS)
- **GIL** → Think: Python's threading bottleneck
- **JWT** → Think: Stateless auth token
- **pprof** → Think: Go's built-in profiler
- **APM** → Think: New Relic/Datadog monitoring
- **p99** → Think: 99% of requests faster than this

Related Documentation

ASGI-VS-GOLANG.md - Full comparison CONCURRENCY-MODELS.md - Technical deep dive PRODUCTION-CONCERNS.md - Security, monitoring, auth **Questions?** All terms explained in context throughout the documentation!